If we move to $\mathbb{R}^3$, then the possible neighbors for each cube are 26, but the minimum distance will be $\sqrt[3]{1/n}$, with similar considerations we can infer that only less than 8 regions will be close enough to have points within the minimum distance found so far; likewise, if we move to $\mathbb{R}^4$ etc...

So, in summary, after finding the region where the target point of our search lies, we need to examine another `O(2k)` points, making total running time `O(2k + log(n))`. While `k` is a constant for a given dataset, and we could in theory ignore it in big-o analysis, in the general case `k` is considered a parameter, as we measure the performance of this method with respect to both size and dimension of the k-d tree changes; it's also apparent that for large values of `k`, `2k` becomes so big that it dominates the running time, since

$$\log(n) > 2^k \Leftrightarrow n > 2^{2^k}$$

and in practice for $k \geq 7$ there is already no chance to have a dataset big enough to satisfy the inequality above.

For `n > 2k`, however, this method has still an advantage on the brute-force search.

A final consideration: pruning heavily depends on the "quality" of the nearest neighbor we have found so far. The shortest the distance to current nearest neighbor, the more branches we can prune, and in turn the highest speed-up we can get. Therefore, it is important to update this value as soon as possible (we do this, in our code, for each node on the first time we visit it) and to traverse the most promising